Programmierbeleg - Bild Metadaten Analyzer

Frau
Melissa Futtig

Matrikelnummer:
107813

Seminargruppe:
CC24w1-B

Modulbezeichnung:
Programmierung II

Datum:
13. März 2026

Inhaltsverzeichnis

1. Einleitung **1**

2. Aufgabenstellung **2**

3. Ziel des Programms **3**
3.1. EXIF-Inhalte . . . 3
3.2. Einsatzzweck . . . 3

4. Anleitungen **5**
4.1. Installationsanleitung . . . 5
4.1.1. Installation von Bibliotheken . . . 5
4.2. Konfigurationsanleitung . . . 7
4.3. Bedienungsanleitung . . . 7

5. Programmaufbau **10**
5.1. Vorstellung Quellcode-Dateien . . . 10
5.1.1. main.py . . . 10
5.1.2. export.py . . . 10
5.1.3. metadaten.py . . . 10
5.1.4. reader.py . . . 11
5.2. Programmablauf . . . 11

6. Fazit **12**
6.1. Verwendete Hilfsmittel . . . 12
6.2. Technische Herausforderungen . . . 12
6.3. Weitere Features . . . 12

A. Anhang **13**

Literatur **I**

1. Einleitung

Die vorliegende Belegarbeit wird im Rahmen des Moduls “Programmierung II“ des Studiengangs “Bachelor of Science in IT-Forensik/ Cybercrime“, der Seminargruppe CC24w1-B, an der Hochschule Mittweida erstellt.
Dabei soll ein freigewähltes Programm in der Programmiersprache Python geschrieben werden, das sich erfolgreich implementieren und kompilieren lässt. Dieses trägt den Namen *Bild-Metadaten-Analyzer*. In folgender Aufgabenstellung werden die Kriterien und Anforderungen an das Programm beschrieben. Im Ziel des Programms aus Kapitel 3 werden die Programminhalte näher betrachtet.

2. Aufgabenstellung

Das Programm sollte ...

... in der Programmiersprache Python geschrieben sein.
... mit einer Schnittstelle (GUI oder CLI) ausgestattet sein.
... Inhalte verarbeiten und ausgeben können.
... sich erfolgreich implementieren und kompilieren lassen.
... plattformübergreifend laufend sein (Windows und Linux).
... ohne kostenpflichtige Software oder Bibliothekne auskommen.
... Quellcode mit beschreibenden Kommentaren enthalten.
... gegebenenfalls Dateien enthalten, die zu Testzwecken genutzt werden können.

Durch die freie Themenwahl gab es eine Vielzahl an Möglichkeiten, passende Programme zu erstellen. Damit dieses einen Bezug zum Studiengang hat, sollte es IT-forensische Inhalte beinhalten. So wurde sich für das Programm *Bild-Metadaten-Analyzer* entschieden, das die Metadaten von Bildern analysieren und ausgeben kann. Es soll dabei helfen, Informationen über die Bilder zu erhalten, die für die IT-Forensik relevant sein können.

3. Ziel des Programms

Damit das Programm *Bild-Metadaten-Analyzer* neben den Anforderungen aus der Aufgabenstellung erfolgreich läuft, sind weitere Kriterien festegelegt worden.
Unter anderem soll es ...

... eine forensische Analyse von Bilddateien (*.jpeg, *.jpg, *.png, *.tif, und *.tiff) durchführen können.
... die ausgelesenen Metadaten bzw. EXIF-Daten übersichtlich darstellen können.
... über eine graphische Oberfläche (GUI), als benutzerfreundliche Schnittstelle, erreichbar sein.
... die gefundenen Metadaten in einer **.csv-Datei* oder in einer **.json-Datei* extrahieren können.

EXIF-Daten sind Metadaten, die in Bilddateien enthalten sind. EXIF ist ein Akronym und steht für Exchangeable Image File Format. "Dieses Dateiformat ist ein Standardformat und wird zum Speichern von Metadaten in digitalen Bildern verwendet. Diese Metadaten enthalten Informationen über das Bild, etwa die Einstellungen der Kamera wie Belichtungszeit, Blende und ISO-Wert. Aber auch Daten wie Datum, Uhrzeit und verwendete Kamera bei der Aufnahme. Wenn die Kamera mit einem GPS-System ausgestattet ist, können sogar die geografische Daten in den EXIF-Daten gespeichert sein." [1]

3.1. EXIF-Inhalte

Wie in der Definition zu EXIF-Daten aus der Quelle [1, E. M. Zylinski. „Was sind EXIF-Daten?"], werden in dem Programm *Bild-Metadaten-Analyzer* die folgenden EXIF-Inhalte ausgelesen und dargestellt ...
... Dateipfad mit Dateinamen,
... Dateigröße,
... Aufnahmedatum und -uhrzeit,
... Kamerahersteller,
... Kameramodell,
... Objektiv,
... Software,
... Blende,
... Belichtungszeit,
... ISO-Wert,
... Brennweite,
... GPS-Daten.

Wenn EXIF-Daten nicht vorhanden sind, soll eine spezifische Fehlermeldung bzw. Information ausgegeben werden, sodass der Nutzer informiert ist, dass für das Bild keine EXIF-Daten vorliegen.

3.2. Einsatzzweck

Der sogenannte *Bild-Metadaten-Analyzer* kann in der IT-Forensik von Nutzung sein, um Ermittlern der Polizei oder weiteren nachrichtendienstlichen Behörden Informationen über Bilder zu liefern, die in einem Ermittlungsverfahren von Relevanz sein könnten.

Ein Anwendungsbeispiel hierbei wären Bilder, die einen Zeitstempel mit GPS-Daten enthalten. Solche könnten unterstützend wirken, um potentielle Täter oder Opfer eines Tatortes leichter zu identifizieren.

4. Anleitungen

In den folgenden Anleitungen (Installationsanleitung, Konfigurationsanleitung und Bedienungsanleitung) werden die Installation, Konfiguration und Bedienung des Programms *Bild-Metadaten-Analyzer* beschrieben.

4.1. Installationsanleitung

Die Installationsanleitung enthält technische Voraussetzungen für die Installation auf einem MacOS, Windows oder Linux Betriebssystem. Es wird darauf eingegangen, wie das Programm zum "Laufen" gebracht wird.

Technische Voraussetzungen:
- Python 3.10
- Betriebssystem: Windows 10/11, MacOS 10.15 oder höher, Linux (Ubuntu 20.04 oder höher)
- Mindestens 4 GB RAM
- Mindestens 100 MB freier Festplattenspeicher

Hinweis: Das Programm wurde ausschließlich auf den oben genannten Betriebssystemen getestet und für die Entwicklung ein MacBook Pro mit MacOS 15.5 verwendet.

Um Python zu installieren, kann etweder die offizielle Python-Webseite besucht oder folgende Schritte aus Kapitel 4.1.1 Installation von Bibliotheken durchlaufen werden.

Benötigte Bibliotheken:
- Pillow (für die Bildverarbeitung bzw. die Bildübertragungsprozesse)
- Tkinter (für die GUI-Entwicklung und ist oft in der Python-Standardbibliothek enthalten)

Standardbibliotheken
Folgende Standardbibliotheken werden für die Entwicklung für das Programm *Bild-Metadaten-Analyzer* verwendet, kommen jedoch mit der Installation von Python (siehe Kapitel 4.1.1 Installation von Bibliotheken)einher:
- JSON: für die Verarbeitung von JSON-Daten –> zum Speichern von Dateien im JSON-Format
- CSV: für die Verarbeitung von CSV-Daten –> zum Speichern von Dateien im CSV-Format

4.1.1. Installation von Bibliotheken

Installation auf Windows:

Python
Für die Installation von Python auf einem Windows-Betriebssystem wurde die Dokumentation von Python selbst genutzt. Die einzelnen Schritte werden hier nicht näher beschrieben, um den Rahmen der Belegarbeit nicht auszureizen.

TKinter
Die Installation von TKinter unter Windows erfordert Python 3.10 oder höher als Vorinstallation. Diese Bibliothek lässt sich über die Windows Eingabeaufforderung (CMD) mit folgendem Befehl installatieren:

```
pip install python-tk
```

Pillow
Die Installation von Pillow unter Windows erfordert Python 3.10 oder höher als Vorinstallation. Diese Bibliothek lässt sich über die Windows Eingabeaufforderung (CMD) mit folgendem Befehl installatieren:

```
pip install pillow
```

Hinweis: Für den Fall, dass 'pip' nicht erkannt wird, kann auch 'pip3' verwendet werden.

Installation auf Linux:

Python
Für die Installation von Python auf einem Linux-Betriebssystem wurde sich an der Dokumentation von Computing for Geeks für ein Ubuntu 20.04 und höher orientiert. Die Installation erfolgt nicht über die GUI, sondern die Kommandozeile (Terminal).

Mit folgenden Befehelen kann Python installiert werden:

```
sudo apt install software-properties-common -y

sudo add-apt-repository ppa:deadsnakes/ppa

sudo apt install python3.10
```

Mit folgendem Befehl erfolgt die Überprüfung, ob Python erfolgreich installiert wurde:

```
python3.10 --version
```

Hinweis: Wenn Befehle mit 'sudo' ausgeführt werden, sollte das Passwort des Benutzers bereitgehalten und eingegeben werden.

TKinter
Die Installation von TKinter unter Linux erfordert Python 3.10 oder höher als Vorinstallation. Diese Bibliothek lässt sich über das Linux Terminal mit folgendem Befehl installatieren:

```
sudo apt install python3-tk
```

Pillow
Die Installation von Pillow unter Linux erfordert Python 3.10 oder höher als Vorinstallation. Diese Bibliothek lässt sich über das Linux Terminal mit folgendem Befehl installatieren:

```
sudo apt install pillow
```

oder auch

```
pip install pillow
```

4.2. Konfigurationsanleitung

Die Konfigurationsanleitung beschreibt welche Einstellungen als Nutzer selbst vorgenommen werden könnten. Als mögliche Einstellungen werden unter anderem die Bildformate betrachtet, die ausschließlich analysiert werden können. Dabei handelt es sich um die Formate, welche bereits in Kapitel 3 Ziel des Programms gelistet wurden.

Des Weiteren hat der Nutzer die Wahl, ob die ausgelesenen Metadaten als **.csv-Datei* oder als **.json-Datei* extrahiert werden sollen. Der Speicherort kann hierbei **nicht** frei gewählt werden und ist im Programm-Code festgelegt und befindet sich **immer** im DOWNLOADS-Ordner eines jeden Systems.

Zusätzlich ist die Namensgebung der extrahierten Dateien festgelegt. Die Dateien behalten ihren ursprünglichen Dateinamen, jedoch mit der Endung **_Metadata_Auswertung.json* oder **_Metadata_Auswertung.csv*, je nachdem, für welches Format sich der Nutzer entscheidet.

Die Ausgabe der Sprache lässt sich nicht verändern. Die Originalsprache des Systems der Bilder ist englisch und wird in das Deutsche übersetzt.

4.3. Bedienungsanleitung

In wenigen Schritten ist es einem Endnutzer diesen Programmes möglich dieses auszuführen.

Schritt 1:
Das Programm *Bild-Metadaten-Analyzer* wird über die ausführbare Datei *main.py* gestartet. Ein Weg wäre es über die Eingabeaufforderung (CMD) unter Windows oder über das Terminal unter Linux auszuführen ODER über eine Intelligente Entwicklungsumgebung (IDE) wie zum Beispiel IntelliJ oder Visual Studio Code.

Gestartet wird das Programm mit folgendem Befehl:

```
python main.py
```

Alternativ auch mit:

```
python3 main.py
```

Schritt 2:
Nach dem Starten des Programms öffnet sich die graphische Oberfläche (GUI) des Programms. Es erscheint ein Fenster, in dem der Endnutzer auswählen kann, welche Bilddatei betrachtet werden soll. Alle Dateien, die nicht kompatibel sind, werden automatisch ausgegraut, sodass eine Auswahl dieser nicht erfolgen kann.

Abbildung 1: GUI des Programms *Bild-Metadaten-Analyzer*

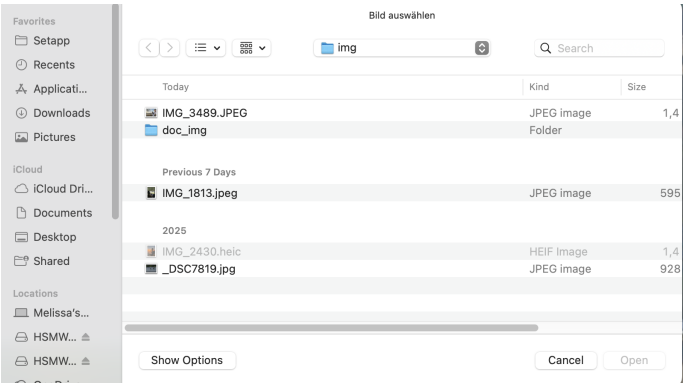


Abbildung 2: Auswahl des Bildes *Bild-Metadaten-Analyzer*

Schritt 3:
Nachdem sich für eine Bilddatei entschieden wurde, liest das Programm die EXIF-Daten aus und präsentiert das Ergebnis dem Endnutzer.

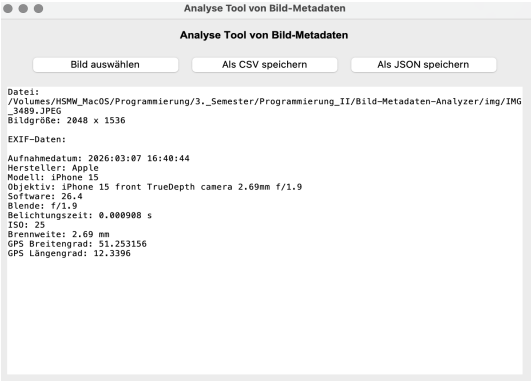


Abbildung 3: Ausgabe der vorhandenen Bild-Metdadaten

Schritt 4:
Im Anschluss hat der Endnutzer die Wahl, ob die ausgelesenen Metadaten in einer **.csv-Datei* oder in einer **.json-Datei* extrahiert werden sollen. Entscheidet der Endnutzer sich für eine *csv-Datei, so wird eine Datei mit der Endung **_Metadata_Auswertung.csv* im DOWNLOADS-Ordner des Systems erstellt. Entscheidet er sich für eine *json-Datei, so wird eine Datei mit der Endung **_Metadata_Auswertung.json* im DOWNLOADS-Ordner des Systems erstellt.
Nach der Extrahierung der Metadaten in entweder einer **.csv-Datei* oder einer **.json-Datei* wird eine Bestätigungsmeldung ausgegeben, dass die Datei erfolgreich erstellt wurde.

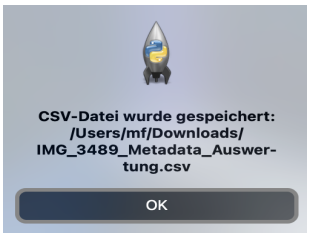


Abbildung 4: Speicherung als *.csv-Datei

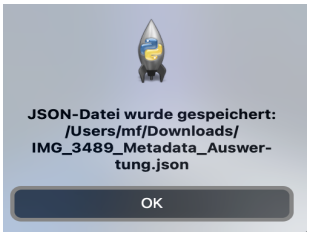


Abbildung 5: Speicherung als *.json-Datei

Schritt 5:
Die Datei wurde im Downloads-Ordner des Systems abgelegt und kann dort eingesehen und geöffnet werden.

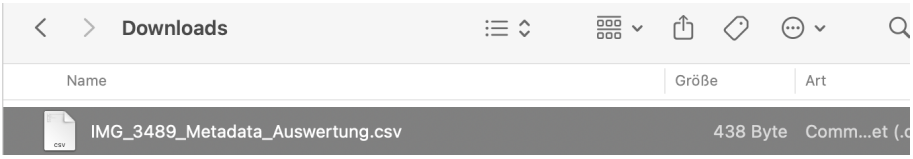
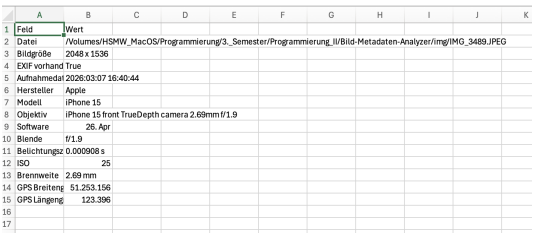


Abbildung 6: csv-Datei im Downloads-Ordner

Schritt 6:
Die csv-Datei öffnet sich automatisch mit Excel. Die json-Datei öffnet sich automatisch mit dem Standard-Texteditor des Systems.



	A	B	C	D	E	F	G	H	I	J	K
1	Feld	Wert									
2	Datei	/Volumes/HSMW_MacOS/Programmierung/3._Semester/Programmierung_II/Bild-Metadaten-Analyzer/img/IMG_3489.JPEG									
3	Bildgröße	2048 x 1536									
4	EXIF vorhand	True									
5	Aufnahmedat	2026:03:07 16:40:44									
6	Hersteller	Apple									
7	Modell	iPhone 15									
8	Objektiv	iPhone 15 front TrueDepth camera 2.69mm f/1.9									
9	Software	26. Apr									
10	Blende	f/1.9									
11	Belichtungsz	0.000908 s									
12	ISO	25									
13	Brennweite	2.69 mm									
14	GPS Breiteng	51.253.156									
15	GPS Längeng	123.396									
16											
17											

Abbildung 7: csv-Datei geöffnet mit Excel

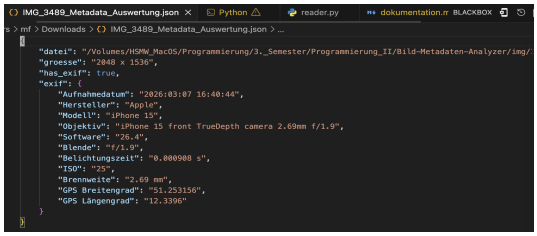


Abbildung 8: json-Datei geöffnet mit Visual Studio Code

5. Programmaufbau

5.1. Vorstellung Quellcode-Dateien

Das Programm ist in vier Python Quellcode-Dateien aufgeteilt, wobei jede Datei eine individuelle Rolle einnimmt. Durch diese Struktur wird eine klare Trennung zwischen Benutzeroberfläche, Datenverarbeitung und Datenexport erreicht.

5.1.1. main.py

Die Hauptdatei des Programms ist die *main.py*. In dieser Datei befindet sich ausschließlich der Programmstart. Diese Datei ruft die Funktion *starte_app* auf, welche sich in der *metadaten.py*-Datei befindet.
Zu beachten hierbei ist, dass in der *main.py* die Datei *metadaten.py* erwähnt wird. Ansonsten lassen sich die Funktionen nicht verwenden.

Aufbau der *main.py*:

```
from metadaten import starte_app

if __name__ == "__main__":
    starte_app()
```

5.1.2. export.py

Die Datei *export.py* ist für den Export der Metadaten verantwortlich. Die ausgelesenen Informationen können hier als CSV- oder JSON-Datei gespeichert werden.
Folgende Funktionen sind nochmal genauer beschrieben:

- *als_csv_exportieren*: Exportiert die Metadaten bzw. EXIF-Daten in eine CSV-Datei.
- *als_json_exportieren*: Speichert die Metadaten in einer JSON-Datei.

5.1.3. metadaten.py

Die Datei *metadaten.py* beinhaltet enthält Funktionen zur Nutzung der GUI, als auch zur Verarbeitung von Benutzereingaben. Dazu gehören unter Anderem das Öffnen von Bilddateien, das Anzeigen der Metadaten oder das Speichern der Daten in verschiedenen Formaten.
Folgende Funktionen sind nochmal genauer beschrieben:

- *bild_oeffnen*: Öffnet eine ausgewählte Bilddatei über das Startfeld und startet anschließend die Analyse der Metadaten.
- *metadaten_text_formatieren*: Formatiert die ausgelesenen Metadaten, sodass sie übersichtlich im Textfeld der Benutzeroberfläche angezeigt werden können.
- *datei_wird_unterstuetzt*: Überprüft, ob das ausgewählte Bildformat vom Programm unterstützt wird.
- *csv_speichern*: Speichert die ausgelesenen Metadaten als CSV-Datei.
- *json_speichern*: Speichert die ausgelesenen Metadaten als JSON-Datei.

5.1.4. reader.py

In der Datei *reader.py* befindet sich die eigentliche Logik zum Auslesen der Bildmetadaten. Mithilfe der Bibliothek Pillow werden EXIF-Daten aus den Bilddateien extrahiert und in einer strukturierten Form gespeichert.
Folgende Funktionen sind nochmal genauer beschrieben:

- *bild_metadaten_auslesen*: Öffnet eine Bilddatei und extrahiert relevante Metadaten wie Aufnahmedatum, Kamerahersteller, Kameramodell, Blende, Belichtungszeit, ISO-Wert und Brennweite.
- *gps_daten_auslesen*: Liest vorhandene GPS-Daten aus den EXIF-Metadaten und berechnet daraus die geografischen Koordinaten.
- *grad_umrechnen*: Wandelt GPS-Koordinaten aus Grad-Minuten-Sekunden in Dezimalgrad um.

5.2. Programmablauf

Der Programmablauf orientiert sich an der Bedienungsanleitung aus Kapitel 4.3 Bedienungsanleitung.

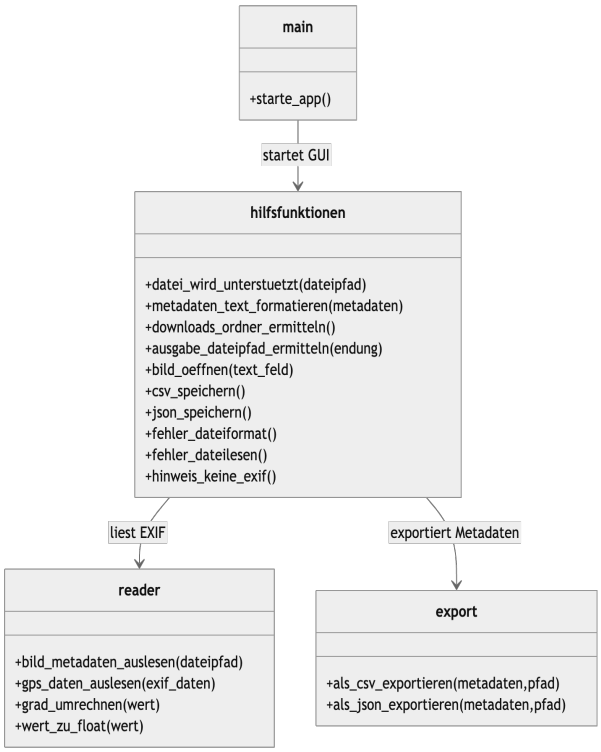


Abbildung 9: Klassendiagramm des Programms *Bild-Metadaten-Analyzer*

6. Fazit

dummytext

6.1. Verwendete Hilfsmittel

6.2. Technische Herausforderungen

6.3. Weitere Features

Überprüfung, ob die EXIF-Inhalte verändert worden sind

A. Anhang

Listing 1: Python Main-Datei

```
from metadaten import starte_app


if __name__ == "__main__":
    starte_app()
```

```
from PIL import Image, ExifTags
```

Einverständniserklärung

```
def wert_zu_float(wert):
    try:
        return float(wert)
    except:
        try:
            return float(wert[0]) / float(wert[1])
        except:
            return None


def bruch_als_text(wert):
    try:
        if hasattr(wert, "numerator") and hasattr(wert, "denominator"):
            return str(wert.numerator) + "/" + str(wert.denominator)

        if isinstance(wert, tuple) and len(wert) == 2:
            return str(wert[0]) + "/" + str(wert[1])

        return str(wert)
    except:
        return str(wert)


def grad_umrechnen(wert):
    if len(wert) < 3:
        return None

    grad = wert_zu_float(wert[0])
    minuten = wert_zu_float(wert[1])
    sekunden = wert_zu_float(wert[2])

    if grad is None or minuten is None or sekunden is None:
        return None

    return grad + (minuten / 60.0) + (sekunden / 3600.0)


def gps_daten_auslesen(exif_daten):
    breitengrad = None
    laengengrad = None

    gps_ifd = None

    try:
        gps_ifd = exif_daten.get_ifd(ExifTags.IFD.GPSInfo)
    except:
        gps_ifd = None

    if not gps_ifd:
        for tag_id in exif_daten:
            tag_name = ExifTags.TAGS.get(tag_id, tag_id)
            if tag_name == "GPSInfo":
                gps_ifd = exif_daten.get(tag_id)
                break

    if not gps_ifd:
        return None, None

    gps_daten = {}

    for schluessel in gps_ifd:
        name = ExifTags.GPSTAGS.get(schluessel, schluessel)
        gps_daten[name] = gps_ifd[schluessel]

    if "GPSLatitude" in gps_daten and "GPSLatitudeRef" in gps_daten:
        try:
            breitengrad = grad_umrechnen(gps_daten["GPSLatitude"])
            referenz = gps_daten["GPSLatitudeRef"]

            if isinstance(referenz, bytes):
                referenz = referenz.decode(errors="ignore")

            if referenz != "N":
                breitengrad = -breitengrad
        except:
```

```
            breitengrad = None
```

Einverständniserklärung

```
    if "GPSLongitude" in gps_daten and "GPSLongitudeRef" in gps_daten:
        try:
            laengengrad = grad_umrechnen(gps_daten["GPSLongitude"])
            referenz = gps_daten["GPSLongitudeRef"]

            if isinstance(referenz, bytes):
                referenz = referenz.decode(errors="ignore")

            if referenz != "E":
                laengengrad = -laengengrad
        except:
            laengengrad = None

    return breitengrad, laengengrad


def bild_metadaten_auslesen(dateipfad):
    metadaten = {}

    metadaten["datei"] = dateipfad
    metadaten["groesse"] = "Nicht vorhanden"
    metadaten["has_exif"] = False
    metadaten["exif"] = {}

    metadaten["exif"]["Aufnahmedatum"] = "Nicht vorhanden"
    metadaten["exif"]["Hersteller"] = "Nicht vorhanden"
    metadaten["exif"]["Modell"] = "Nicht vorhanden"
    metadaten["exif"]["Objektiv"] = "Nicht vorhanden"
    metadaten["exif"]["Software"] = "Nicht vorhanden"
    metadaten["exif"]["Blende"] = "Nicht vorhanden"
    metadaten["exif"]["Belichtungszeit"] = "Nicht vorhanden"
    metadaten["exif"]["ISO"] = "Nicht vorhanden"
    metadaten["exif"]["Brennweite"] = "Nicht vorhanden"
    metadaten["exif"]["GPS Breitengrad"] = "Nicht vorhanden"
    metadaten["exif"]["GPS L√§ngengrad"] = "Nicht vorhanden"

    bild = Image.open(dateipfad)

    metadaten["groesse"] = str(bild.size[0]) + " x " + str(bild.size[1])

    exif_daten = bild.getexif()

    if exif_daten:
        exif_gefunden = False

        for tag_id in exif_daten:
            tag_name = ExifTags.TAGS.get(tag_id, tag_id)
            wert = exif_daten.get(tag_id)

            if tag_name == "DateTimeOriginal":
                metadaten["exif"]["Aufnahmedatum"] = str(wert)
                exif_gefunden = True

            elif tag_name == "DateTime" and metadaten["exif"]["Aufnahmedatum"] == "Nicht
vorhanden":
                metadaten["exif"]["Aufnahmedatum"] = str(wert)
                exif_gefunden = True

            elif tag_name == "Make":
                metadaten["exif"]["Hersteller"] = str(wert)
                exif_gefunden = True

            elif tag_name == "Model":
                metadaten["exif"]["Modell"] = str(wert)
                exif_gefunden = True

            elif tag_name == "Software":
                metadaten["exif"]["Software"] = str(wert)
                exif_gefunden = True

        try:
            exif_ifd = exif_daten.get_ifd(ExifTags.IFD.Exif)
        except:
            exif_ifd = {}

        if exif_ifd:
```

Einverständniserklärung

```
            for tag_id in exif_ifd:
                tag_name = ExifTags.TAGS.get(tag_id, tag_id)
                wert = exif_ifd[tag_id]

                if tag_name == "LensModel":
                    metadaten["exif"]["Objektiv"] = str(wert)
                    exif_gefunden = True

                elif tag_name == "FNumber":
                    blende = wert_zu_float(wert)
                    if blende is not None:
                        metadaten["exif"]["Blende"] = "f/" + str(round(blende, 2))
                    exif_gefunden = True

                elif tag_name == "ExposureTime":
                    belichtungszeit = wert_zu_float(wert)
                    if belichtungszeit is not None:
                        metadaten["exif"]["Belichtungszeit"] = str(round(belichtungszeit, 6)) +
" s"
                    else:
                        metadaten["exif"]["Belichtungszeit"] = bruch_als_text(wert) + " s"
                    exif_gefunden = True

                elif tag_name == "ISOSpeedRatings":
                    metadaten["exif"]["ISO"] = str(wert)
                    exif_gefunden = True

                elif tag_name == "PhotographicSensitivity":
                    metadaten["exif"]["ISO"] = str(wert)
                    exif_gefunden = True

                elif tag_name == "FocalLength":
                    brennweite = wert_zu_float(wert)
                    if brennweite is not None:
                        metadaten["exif"]["Brennweite"] = str(round(brennweite, 2)) + " mm"
                    exif_gefunden = True

        breitengrad, laengengrad = gps_daten_auslesen(exif_daten)

        if breitengrad is not None:
            metadaten["exif"]["GPS Breitengrad"] = str(round(breitengrad, 6))

        if laengengrad is not None:
            metadaten["exif"]["GPS L√§ngengrad"] = str(round(laengengrad, 6))

        if exif_gefunden:
            metadaten["has_exif"] = True

    bild.close()

    return metadaten
```

Literatur

[1] E. M. Zylinski. „Was sind EXIF-Daten?“ (12. Juli 2023), Adresse: https://www.heise.de/tipps-tricks/Was-sind-EXIF-Daten-9213830.html#top (besucht am 13. 03. 2026).